

Course: Foundations of AI with Embeddings

Lecture 2: tSNE

Liubov B.T., Vladimir K.

Constructor University, March 9, 2026

Notations we use for tSNE method

- ▶ High-dimensional points: $x_i \in \mathbb{R}^D$, $i = 1, \dots, N$.
- ▶ We want a low-dimensional embedding $y_i \in \mathbb{R}^d$ (typically $d = 2$) that preserves local neighborhoods.
- ▶ We would like to overcome '**crowding problem**', hence **non-linear methods** are needed.

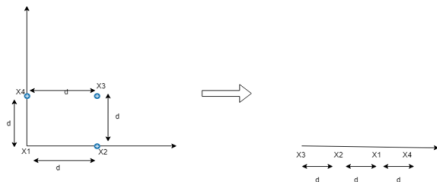
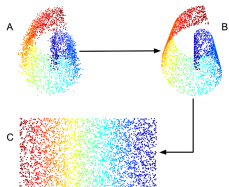


Figure 8 – Crowding problem

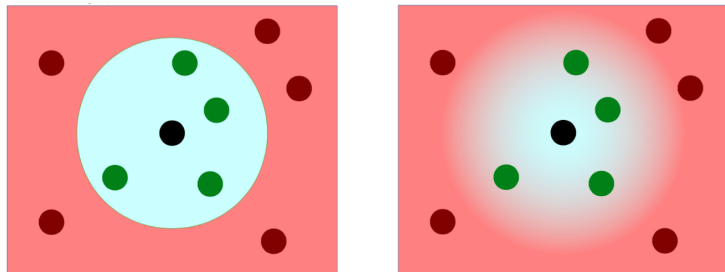
Notations we use for tSNE method

- ▶ High-dimensional points: $x_i \in \mathbb{R}^D$, $i = 1, \dots, N$.
- ▶ We want a low-dimensional embedding $y_i \in \mathbb{R}^d$ (typically $d = 2$) that preserves local neighborhoods.
- ▶ We need to keep low-dimensional representations of dissimilar datapoints far apart. Methods like (PCA; Hotelling, 1933) and (MDS; Torgerson, 1952) are linear (**see the lecture 1**). For projection of high-dimensional data on a low-dimensional, we need to keep the low-dimensional representations of very similar datapoints on data manifold close.



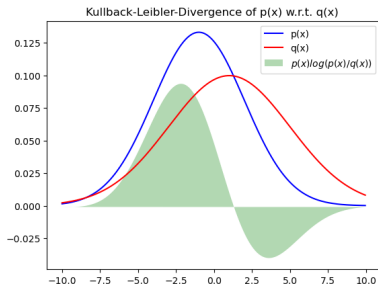
t-SNE: high-level idea

- ▶ Interpret pairwise similarities in high dimension as probabilities p_{ij} : preserving **soft neighborhood structure**.
- ▶ Find low-dimensional points with probabilities q_{ij} that match p_{ij} using cost function C : space $X \rightarrow$ space Y .
- ▶ Optimize function C by minimizing a Kullback–Leibler (KL) divergence $KL(P\|Q)$.



t-SNE: high-level idea

- ▶ Interpret pairwise similarities in high dimension as probabilities p_{ij} : preserving **soft neighborhood structure**.
- ▶ Find low-dimensional points with probabilities q_{ij} that match p_{ij} using cost function C : space $X \rightarrow$ space Y .
- ▶ Optimize function C by minimizing a Kullback–Leibler (KL) divergence $KL(P\|Q)$.



High-dimensional conditional similarities $p_{j|i}$

Define squared distances $d_{ij} = \|x_i - x_j\|^2$. For each i , we use a Gaussian kernel:

$$p_{j|i} = \frac{\exp\left(-\frac{d_{ij}}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{d_{ik}}{2\sigma_i^2}\right)}, \quad p_{i|i} = 0.$$

Closer points in high dimension $\Rightarrow$ larger $p_{j|i}$.

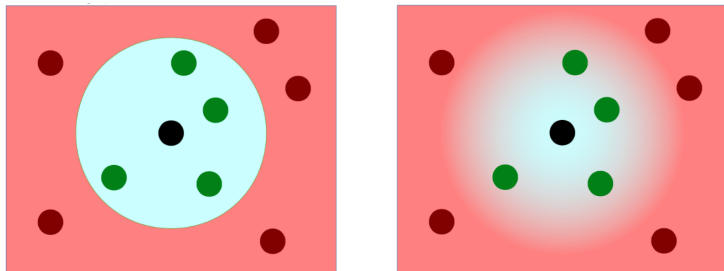
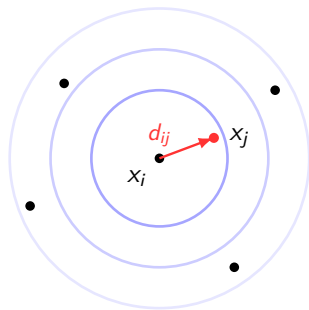


Illustration of $p_{j|i}$ (soft neighborhoods around x_i)

- ▶ Around each point x_i we place a Gaussian with variance σ_i^2 .
- ▶ Nearby points x_j have higher probability $p_{j|i}$.
- ▶ σ_i controls the local scale (later set via perplexity hyperparameter). This measures how close x_j is from x_i , considering a Gaussian distribution around x_i with a given variance σ_i^2 . The variance is different for every point; it is chosen such that points in dense areas are given a smaller variance than points in sparse areas. The original paper details how this variance is computed exactly.



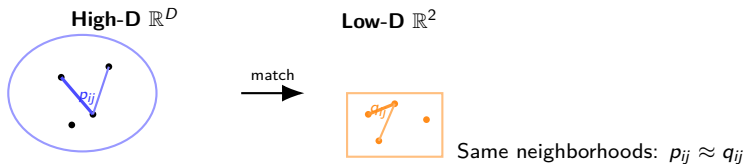
Symmetric joint probabilities p_{ij}

For **tSNE method** we symmetrize and normalize:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}, \quad p_{ii} = 0,$$

so that

$$\sum_i \sum_j p_{ij} = 1.$$



High-dimensional similarities $p_{j|i} \rightarrow$ Low-dimensional similarities q_{ij}

In the embedding space Y we use a Student t-distribution (1 degree of freedom, heavy tails):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}, \quad q_{ii} = 0,$$

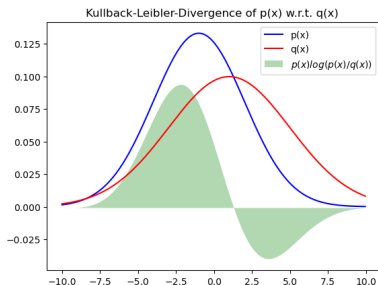
with normalization

$$\sum_i \sum_j q_{ij} = 1.$$

Objective: KL divergence

We match low-dimensional similarities $Q = \{q_{ij}\}$ to high-dimensional ones $P = \{p_{ij}\}$ by minimizing

$$C = KL(P\|Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$



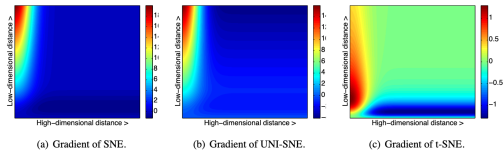
Gradient of the cost function

The gradient of C with respect to y_i is

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij}) \frac{(y_i - y_j)}{1 + \|y_i - y_j\|^2}.$$

- ▶ If $p_{ij} > q_{ij}$: attraction between y_i and y_j .
- ▶ If $p_{ij} < q_{ij}$: repulsion between y_i and y_j .

tSNE: minimizes the KL divergence using gradient descent.



Optimization procedure

Typical gradient descent update (with learning rate η and momentum α):

$$y_i^{(t+1)} = y_i^{(t)} - \eta \frac{\partial \mathcal{C}}{\partial y_i^{(t)}} + \alpha (y_i^{(t)} - y_i^{(t-1)}).$$

Often used in practice:

- ▶ **Early exaggeration:** temporarily multiply p_{ij} to form tight clusters.
- ▶ Learning rate schedule and momentum.

tSNE hyperparameters: perplexity and bandwidth σ_i

Entropy of the conditional distribution $P_i = \{p_{j|i}\}_j$:

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}.$$

Perplexity (effective number of neighbors):

$$\text{Perp}(P_i) = 2^{H(P_i)}.$$

We choose σ_i so that $\text{Perp}(P_i)$ matches a user-specified value. Perp balances attention between local and global aspects of your data; guess the # of close neighbor each point.

Summary of tSNE method

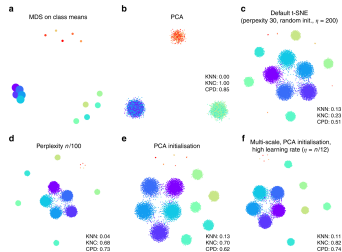
- ▶ High-D similarities p_{ij} from Gaussian kernels of soft neighborhoods.
- ▶ Low-D similarities q_{ij} from a Student t-distribution.
- ▶ Minimize $KL(P||Q)$ to preserve local structure, yet to consider meso-scale of neighborhoods.
- ▶ Optimization uses attractive/repulsive forces between points.

Summary of tSNE method

- ▶ High-D similarities p_{ij} from Gaussian kernels of soft neighborhoods.
- ▶ Low-D similarities q_{ij} from a Student t-distribution.
- ▶ Minimize $KL(P||Q)$ to preserve local structure, yet to consider meso-scale of neighborhoods.
- ▶ Optimization uses attractive/repulsive forces between points.
- ▶ Local nature of t-SNE makes it sensitive to intrinsic dimension of the data and not guaranteed to converge to global minimum.

Limitations of tSNE method

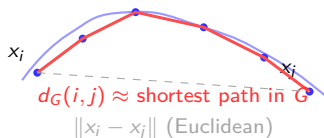
- ▶ A major weakness of t-SNE is that the cost function is not convex, as a result of which several optimization parameters need to be chosen.
- ▶ t-SNE reduces the dimensionality of data mainly based on local properties of the data, which makes t-SNE sensitive to the curse of the intrinsic dimensionality of the data (Bengio, 2007).



- ▶ Limitation of tSNE to 3D embedding in Y space.]

ISOMAP: main idea

- ▶ Nonlinear dimensionality reduction preserving **geodesic distances** along the data manifold.
- ▶ Build a neighborhood graph G :
 - ▶ Nodes: data points x_i .
 - ▶ Edges between nearby points (e.g. k -NN).
 - ▶ Edge weights: Euclidean distances $\|x_i - x_j\|$.
- ▶ Approximate geodesic distance $d_G(i, j)$ by shortest path distance in G .



ISOMAP: from distances to embedding

- ▶ Form the matrix of squared geodesic distances $D^{(G)} = (d_G(i,j)^2)_{i,j}$.
- ▶ Apply classical MDS to $D^{(G)}$:
 - ▶ Double-center: $B = -\frac{1}{2}JD^{(G)}J$, where $J = I - \frac{1}{N}\mathbf{e}\mathbf{e}^T$.
 - ▶ Eigen-decompose: $B = V\Lambda V^T$.
 - ▶ Low-dimensional embedding: rows of $Y = V_d\Lambda_d^{1/2}$ (top d eigenvalues).
- ▶ Result: Y preserves manifold **geodesic distances** in low dimension.